ENTERPRISE JAVA
(25MCA202)

BY

Anantha Murthy
Assistant Professor
Dept. of MCA
NMAM.I.T, Nitte

Unit 3

1. The Collection Framework

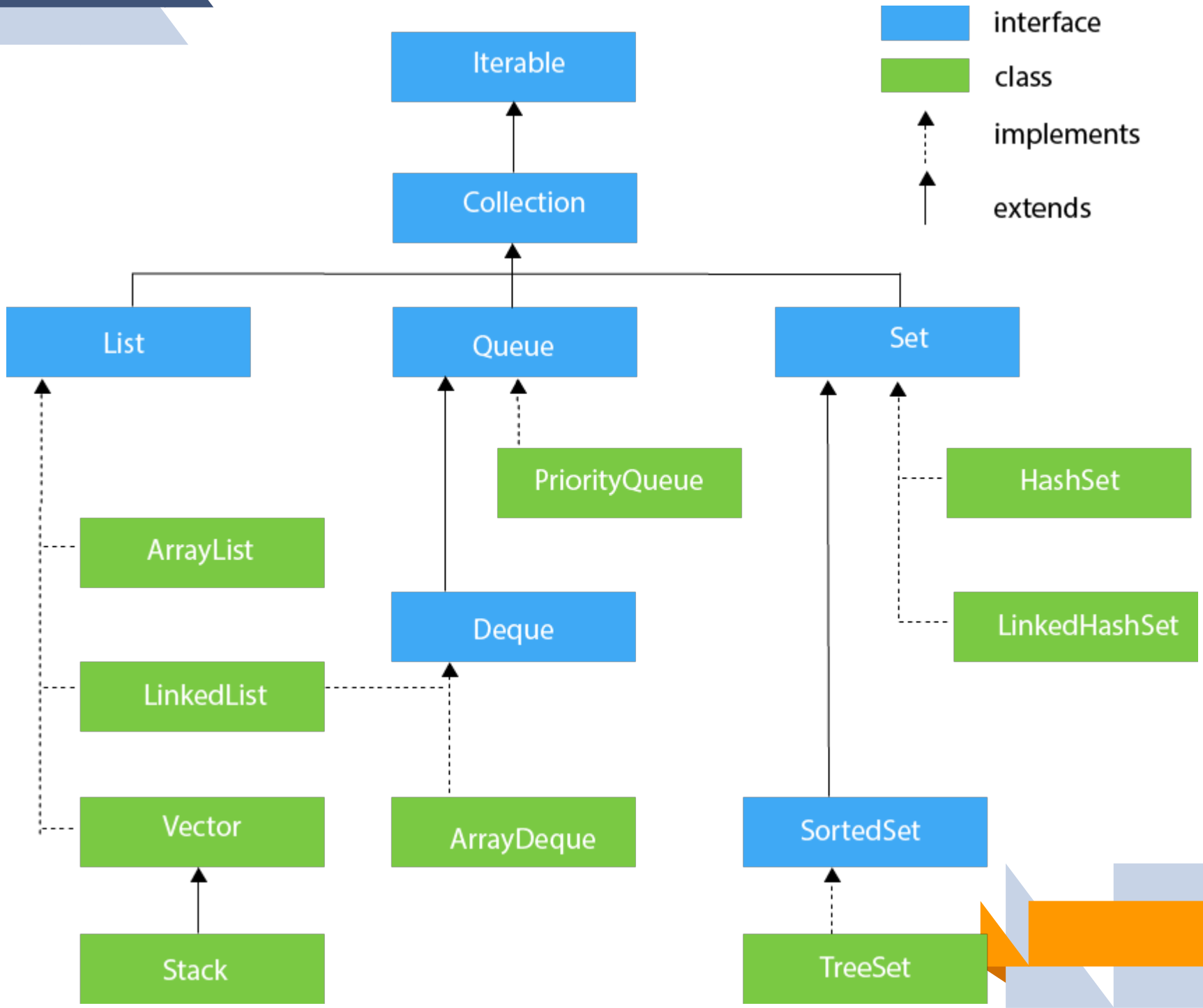
The Collection Framework

- The Collection Framework in Java is a framework that provides an architecture to store and manipulate the group of objects.
- Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.
- Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).
- The java.util package contains all the classes and interfaces for the Collection framework.

Collection Interface

- The Collection interface is the interface which is implemented by all the classes in the collection framework.
- It declares the methods that every collection will have.
- In other words, we can say that the Collection interface builds the foundation on which the collection framework depends.

The Collection Framework Hierarchy



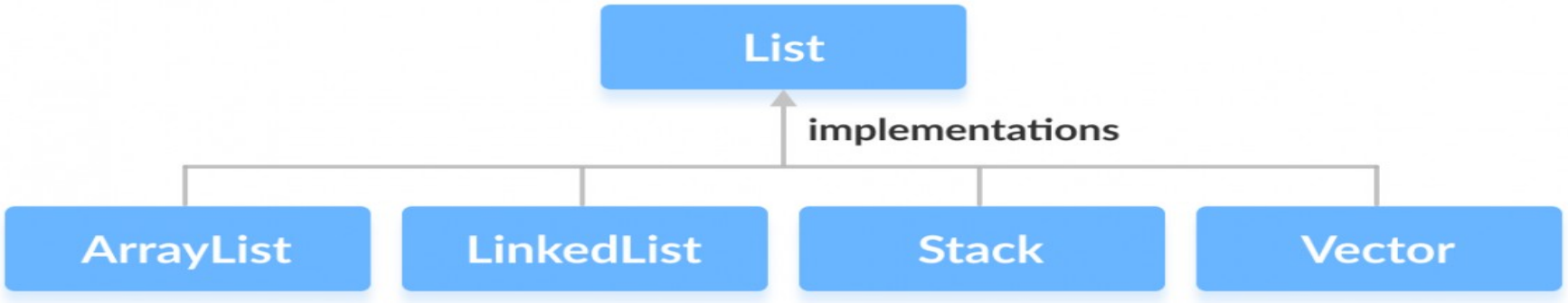
The List Interface

- List interface is the child interface of Collection interface.
- It inhibits a list type data structure in which we can store the ordered collection of objects. It can have duplicate values.
- List interface is implemented by the classes ArrayList, LinkedList, Vector, and Stack.
- To instantiate the List interface, we must use :

  List <data-type> list1= new ArrayList();

  List <data-type> list2 = new LinkedList();

- There are various methods in List interface that can be used to insert, delete, and access the elements from the list.



The List Interface Methods

Methods	Description
add()	adds an element to a list
addAll()	adds all elements of one list to another
get()	helps to randomly access elements from lists
iterator()	returns iterator object that can be used to sequentially access elements of lists
set()	changes elements of lists
remove()	removes an element from the list
removeAll()	removes all the elements from the list
clear()	removes all the elements from the list (more efficient than `removeAll()`)
size()	returns the length of lists
toArray()	converts a list into an array
contains()	returns `true` if a list contains specific element

The ArrayList Class

- The ArrayList class implements the List interface.
- It uses a dynamic array to store the duplicate element of different data types.
- The ArrayList class maintains the insertion order and is non-synchronized.
- The elements stored in the ArrayList class can be randomly accessed.

Operations in a Java ArrayList Interface

- **Operation 1:** Adding elements to List class using add() method
- **Operation 2:** Updating elements in List class using set() method
- **Operation 3:** Searching for elements using indexOf(), lastIndexOf methods
- **Operation 4:** Removing elements using remove() method
- **Operation 5:** Accessing Elements in List class using get() method
- **Operation 6:** Checking if an element is present in the List class using contains() method

The ArrayList Class

```java
import java.util.ArrayList;
import java.util.Iterator;

public class ArrayListDemo {
    public static void main(String[] args) {
        // Creating an ArrayList
        ArrayList<String> list = new ArrayList<>();

        // Adding elements using add()
        list.add("Java");
        list.add("Python");
        list.add("C++");
        list.add("JavaScript");
        list.add(3,"Enterprise Java");
        list.add("Java");
        System.out.println("The index of C++ is "+ list.indexOf("C++"));
        System.out.println("The last index of Java is "+ list.lastIndexOf("Java"
            ));

        // Adding a collection using addAll()
        ArrayList<String> anotherList = new ArrayList<>();
        anotherList.add("Ruby");
        anotherList.add("Swift");
        list.addAll(anotherList);
```

The ArrayList Class

```
// Accessing elements using get()
System.out.println("Element at index 2: " + list.get(2));
// Iterating over elements using iterator()
System.out.println("Iterating over elements:");
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
// Modifying an element using set()
list.set(3, "TypeScript");
 System.out.println("Does the list contain 'C++'? " + list.contains("C++"
     ));
list.remove("C++");  // Removing an element using remove()

list.removeAll(anotherList); // Removing a collection using removeAll()

list.clear(); // Clearing the ArrayList using clear()

// Checking the size of the ArrayList using size()
System.out.println("Size of the ArrayList: " + list.size());

// Converting ArrayList to Array using toArray()
String[] array = list.toArray(new String[0]);

// Checking if an element exists using contains()
System.out.println("Does the list contain 'Java'? " + list.contains("Java"
    ));  }}
```

The ArrayList Class

Output

```
The index of C++ is 2
The last index of Java is 5
Element at index 2: C++
Iterating over elements:
Java
Python
C++
Enterprise Java
JavaScript
Java
Ruby
Swift
Does the list contain 'C++'? true
Size of the ArrayList: 0
Does the list contain 'Java'? false
```

The LinkedList Class

- LinkedList implements the Collection interface.
- It uses a doubly linked list internally to store the elements.
- It can store the duplicate elements.
- It maintains the insertion order and is not synchronized.

```
import java.util.LinkedList;
import java.util.Iterator;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> linkedList = new LinkedList<>();

        // Adding elements using add()
        linkedList.add("Java");
        linkedList.add("Python");
        linkedList.add("C++");
        linkedList.add("JavaScript");
```

The LinkedList Class

```
        // Adding a collection using addAll()
        LinkedList<String> anotherList = new LinkedList<>();
        anotherList.add("Ruby");
        anotherList.add("Swift");
        linkedList.addAll(anotherList);

        System.out.println("Element at index 2: " + linkedList.get(2));
        System.out.println("Iterating over elements:");
        Iterator<String> iterator = linkedList.iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }
        linkedList.set(3, "TypeScript");
        linkedList.remove("C++");
        linkedList.removeAll(anotherList);
        linkedList.clear();

        System.out.println("Size of the LinkedList: " + linkedList.size());
        String[] array = linkedList.toArray(new String[0]);
        System.out.println("Does the list contain 'Java'? " + linkedList.contains
            ("Java"));
    }
}
```

The LinkedList Class

Output

```
Element at index 2: C++
Iterating over elements:
Java
Python
C++
JavaScript
Ruby
Swift
Size of the LinkedList: 0
Does the list contain 'Java'? false
```

The LinkedList Class Vs ArrayList class

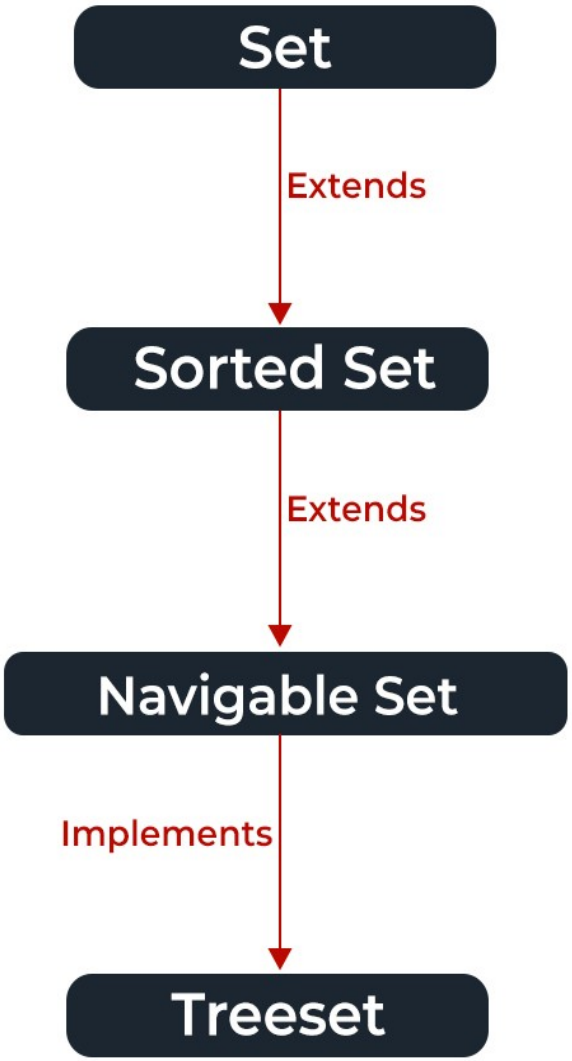
1. **Underlying Data Structure**:
   - **LinkedList**: Internally implemented as a doubly-linked list where each element is stored as a separate object (node) containing a reference to the previous and next elements in the list.
   - **ArrayList**: Internally implemented as a dynamic array that can dynamically resize itself when elements are added or removed.
2. **Performance**:
   - **LinkedList**: Efficient for frequent insertions and deletions at arbitrary positions in the list, as it involves only adjusting the references of neighboring nodes. However, accessing elements by index is less efficient compared to ArrayList because it requires traversing the list from the beginning or end.
   - **ArrayList**: Efficient for random access and retrieval of elements by index since it uses an array-based data structure. However, insertion and deletion operations at arbitrary positions can be less efficient, especially when resizing the underlying array is required.
3. **Memory Overhead**:
   - **LinkedList**: Each element in the list requires additional memory overhead for storing references to the previous and next elements.
   - **ArrayList**: Generally consumes less memory overhead since it stores elements in a contiguous array, without the need for additional references between elements.

The LinkedList Class Vs ArrayList class

4. **Insertion and Deletion**:
   - **LinkedList**: Provides efficient insertion and deletion operations, especially when performed at the beginning or end of the list or when the position is known.
   - **ArrayList**: Insertion and deletion operations may be less efficient, particularly when performed frequently or at arbitrary positions, as it may require shifting elements within the underlying array.
5. **Iterating Over Elements**:
   - **LinkedList**: Iterating over elements using an iterator can be efficient, especially for sequential traversal.
   - **ArrayList**: Iterating over elements is generally efficient, as it provides direct access to elements using indexes.
6. **Use Cases**:
   - **LinkedList**: Suitable for scenarios requiring frequent insertions and deletions, such as queues or certain types of linked data structures.
   - **ArrayList**: Suitable for scenarios requiring fast random access and retrieval, such as maintaining a collection of elements where elements are frequently accessed by their index.

The Set Interface

- The set interface is present in java.util package and extends the Collection interface.
- It is an unordered collection of objects in which duplicate values cannot be stored.
- It is an interface that implements the mathematical set.
- This interface contains the methods inherited from the Collection interface and adds a feature that restricts the insertion of the duplicate elements.
- There are two interfaces that extend the set implementation namely SortedSet and NavigableSet.
- The class which implements the navigable set is a TreeSet which is an implementation of a self-balancing tree.
- Therefore, this interface provides us with a way to navigate through this tree.



The Set Interface

Method	Description
add(element)	This method is used to add a specific element to the set. The function adds the element only if the specified element is not already present in the set else the function returns False if the element is already present in the Set.
addAll(collection)	This method is used to append all of the elements from the mentioned collection to the existing set. The elements are added randomly without following any specific order.
clear()	This method is used to remove all the elements from the set but not delete the set. The reference for the set still exists.
contains(element)	This method is used to check whether a specific element is present in the Set or not.
containsAll(collection)	This method is used to check whether the set contains all the elements present in the given collection or not. This method returns true if the set contains all the elements and returns false if any of the elements are missing.

The Set Interface

hashCode()	This method is used to get the hashCode value for this instance of the Set. It returns an integer value which is the hashCode value for this instance of the Set.
isEmpty()	This method is used to check whether the set is empty or not.
iterator()	This method is used to return the iterator of the set. The elements from the set are returned in a random order.
remove(element)	This method is used to remove the given element from the set. This method returns True if the specified element is present in the Set otherwise it returns False.
removeAll(collection)	This method is used to remove all the elements from the collection which are present in the set. This method returns true if this set changed as a result of the call.
retainAll(collection)	This method is used to retain all the elements from the set which are mentioned in the given collection. This method returns true if this set changed as a result of the call.
size()	This method is used to get the size of the set. This returns an integer value which signifies the number of elements.
toArray()	This method is used to form an array of the same elements as that of the Set.

The Set Interface - Creating Set Objects

- Since Set is an interface, objects cannot be created of the typeset. We always need a class that extends this list in order to create an object of type HashSet or LinkedHashSet.
- The HashSet class in Java represents a collection that implements the Set interface, providing a data structure that stores unique elements. It achieves uniqueness by using a hash table internally, which allows for fast insertion, deletion, and lookup operations.
- The LinkedHashSet class in Java is similar to HashSet, but it maintains a doubly-linked list running through all of its entries. This linked list defines the iteration ordering, which is the order in which elements were inserted into the set.
- // Obj is the type of the object to be stored in Set and set is an object.
  - Set<Obj> set = new HashSet<Obj> ();
  - Set<Obj> set = new LinkedHashSet<Obj> ();
  -
- Examples: HashSetDemo.java, LinkedHashSetDemo.java

Operations on the Set Interface

- The set interface allows the users to perform the basic mathematical operation on the set.
- Let's take two arrays to understand these basic operations. Let set1 = [1, 3, 2, 4, 8, 9, 0] and set2 = [1, 3, 7, 5, 4, 0, 7, 5]. Then the possible operations on the sets are:

- 1. Intersection: This operation returns all the common elements from the given two sets. For the above two sets, the intersection would be: Intersection = [0, 1, 3, 4]

- 2. Union: This operation adds all the elements in one set with the other. For the above two sets, the union would be: Union = [0, 1, 2, 3, 4, 5, 7, 8, 9]

- 3. Difference: This operation removes all the values present in one set from the other set. For the above two sets, the difference would be: Difference = [2, 8, 9]

  Example: SetExample.java

List Vs Set Interface

List	Set
The List is an ordered sequence.	The Set is an unordered sequence.
List allows duplicate elements	Set doesn't allow duplicate elements.
Elements by their position can be accessed.	Position access to elements is not allowed.
Multiple null elements can be stored.	The null element can store only once.
List implementations are ArrayList, LinkedList, Vector, Stack	Set implementations are HashSet, LinkedHashSet.

ENTERPRISE JAVA (22MCA202)

BY

Anantha Murthy
Assistant Professor
Dept. of MCA
NMAM.I.T, Nitte

Vector Class

- The Vector class implements a growable array of objects.
- Vectors fall in legacy classes, but now it is fully compatible with collections.
- It is found in java.util package and implement the List interface.

- Thread-Safe: All methods are synchronized, making it suitable for multi-threaded environments. However, this can lead to performance overhead in single-threaded scenarios.
- Allows Nulls: Can store null elements.
- Enumeration Support: Provides backward compatibility with Enumeration, a legacy way of iterating over elements.

Vector Class

- The Vector class in Java is a legacy class that was used to implement dynamic arrays before the ArrayList was introduced.
- It is synchronized, which makes it thread-safe but slower compared to ArrayList.

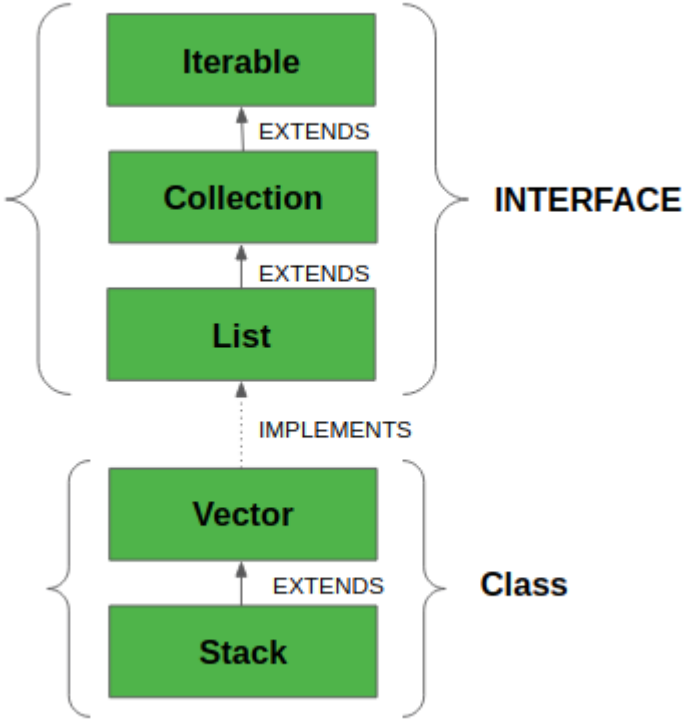
Other Legacy Classes in Java:

- Stack (LIFO data structure)
- Hashtable (Key-value storage, predecessor of HashMap)
- Dictionary (Abstract class, replaced by Map)
- Enumeration (Iterator alternative, now replaced by Iterator)

Even though these classes are still available in Java for backward compatibility, modern Java applications generally prefer newer classes from the Collection framework (like ArrayList, HashMap, etc.) because they are more efficient and flexible.

Vector Class

- They are very similar to ArrayList, but Vector is synchronized and has some legacy methods that the collection framework does not contain.
- It also maintains an insertion order like an ArrayList. Still, it is rarely used in a non-thread environment as it is synchronized, and due to this, it gives a poor performance in adding, searching, deleting, and updating its elements.
- The Iterators returned by the Vector class are fail-fast. In the case of concurrent modification, it fails and throws the ConcurrentModificationException.



Vector Class

Key Features of Vector

Dynamic Size – Unlike arrays, Vector grows automatically when elements are added.

Synchronized – Thread-safe, meaning only one thread can access it at a time.

Can Store Heterogeneous Elements – While typically used for a single type, it can store different data types.

Vector Syntax:

```
import java.util.Vector;
Vector<Type> vectorName = new Vector<>();
```

Type → Data type of elements (e.g., Integer, String, Double, etc.).

vectorName → Name of the vector.

Vector Class

When to Use Vector?

- When thread safety is required (because ArrayList is not synchronized).
- When working with legacy code that already uses Vector.

When Not to Use Vector?

- If performance is a priority, use ArrayList instead because Vector's synchronization adds overhead.
- If thread safety is needed but performance is important, use Collections.synchronizedList(new ArrayList<>()) or CopyOnWriteArrayList.

Vector Class

Sr.No.	Constructor & Description
1	**Vector( )** This constructor creates a default vector, which has an initial size of 10.
2	**Vector(int size)** This constructor accepts an argument that equals to the required size, and creates a vector whose initial capacity is specified by size.
3	**Vector(int size, int incr)** This constructor creates a vector whose initial capacity is specified by size and whose increment is specified by incr. The increment specifies the number of elements to allocate each time that a vector is resized upward.
4	**Vector(Collection c)** This constructor creates a vector that contains the elements of collection c.

Vector Class

Common Methods in Vector

Method	Description
`add(E e)`	Adds an element to the vector.
`add(int index, E e)`	Adds an element at a specified index.
`get(int index)`	Retrieves an element at a specific index.
`size()`	Returns the number of elements in the vector.
`remove(int index)`	Removes an element at a specific index.
`isEmpty()`	Checks if the vector is empty.
`contains(Object o)`	Checks if the vector contains a specific element.
`clear()`	Removes all elements from the vector.

Stack Class

- Java Collection framework provides a Stack class that models and implements a Stack data structure. The class is based on the basic principle of LIFO(last-in-first-out).
- In addition to the basic push and pop operations, the class provides three more functions of empty, search, and peek.
- The Stack class extends Vector and provides additional functionality specifically for stack operations, such as push, pop, peek, empty, and search.
- The Stack class can indeed be referred to as a subclass of Vector, inheriting its methods and properties.

Stack Class

Methods in Stack

Method	Description
`push(E item)`	Adds an item to the top of the stack.
`pop()`	Removes and returns the top element of the stack.
`peek()`	Returns the top element without removing it.
`empty()`	Returns `true` if the stack is empty, otherwise `false`.
`search(Object o)`	Returns the 1-based position of the element in the stack (from top), or `-1` if not found.

Stack Class

In order to create a stack, we must import java.util.stack package and use the Stack() constructor of this class. The below example creates an empty Stack.

```
Stack<E> stack = new Stack<E>();
```

Here E is the type of Object.